胡欣 2023080160

12/23/2025

课程项目 3 ：迷宫宝藏问题

1. 项目概览

本次项目研究“迷宫宝藏问题”。在一个 6×6的迷宫环境中，我用强化学习让 Laura 从起点出发，尽量避开陷阱并走到宝藏。报告里我先把任务的基本要素写清楚，比如状态怎么表示，Laura 能做哪些动作，走一步会怎么变化，以及到达宝藏或踩到陷阱时对应的回报。之后我实现标准的 Q-learning，用 Q 表记录不同位置下各动作的价值，并用 ε-greedy 让模型在探索和利用之间取得平衡，通过多轮训练逐步形成更稳定的策略。在此基础上，我又实现了 2-step Q-learning，让更新时不只看下一步，而是把两步内的回报信息也考虑进去，同时说明它的更新公式、符号含义和更新时机。最后我会对两种方法的学习过程和效果做一个对比，并总结实现过程中遇到的问题和自己的理解。

2. 强化学习建模

为方便后面实现 Q learning 和 2 step Q learning，我先把迷宫环境的状态，动作，状态转移和回报写清楚。

2.1 状态空间

迷宫是 6×6的网格，我用 Laura 当前所在的格子位置表示状态，记为 $S_t = (x_t, y_t)$。其中 $x_t, y_t \in \{0,1,2,3,4,5\}$，因此迷宫里的每个格子都对应一个状态，起点是 $(0,0)$，宝藏所在格子是目标位置，陷阱所在格子是危险位置。当到达宝藏或踩到陷阱时，这一轮探索结束，然后从起点重新开始下一轮。

2.2 动作集合

Laura 每一步只能向四个方向移动一格，动作集合为 $A = \{\text{up,down,left,right}\}$。如果当前位置在边界上，选择会越界的方向时，位置不会改变，仍停留在原来的格子里。

2.3 状态转移概率

迷宫的移动规则是固定的，所以从某个状态 s选择动作 a后，下一步到达的位置是确定的。正常情况下会移动到相邻格子，如果越界就留在原格子，如果移动后到达宝藏则本轮结束，如果移动后进入陷阱则本轮结束，并回到起点开始新一轮。用概率表示就是对任意 s和 a，存在唯一的下一状态 s'，使得 $P(S_{t+1} = s' \mid S_t = s，A_t = a) = 1$，到其他状态的概率为 0。

2.4 回报函数

回报设置为到达宝藏给正回报，踩到陷阱给负回报，其它普通移动为 0，写成分段形式为

$$R_{t+1} = \begin{cases} 1, & \text{到达宝藏。} \\ -1, & \text{踩到陷阱。} \\ 0, & \text{其它情况。} \end{cases}$$

3. Q-learning 算法实现与设计

3.1 程序设计的详细描述

这一部分我主要是把 Q learning 的训练流程加进迷宫程序里，让 Laura 可以通过一轮一轮的尝试，慢慢学出在不同格子里往哪个方向走更合适。为了把这个流程跑通，我重点补了三块内容。第一块是智能体类 QLearningAgent，用来维护 Q 表并决定动作。第二块是训练函数 train_q_learning，用来控制每一轮怎么走，怎么更新，怎么判断训练有没有变稳定。第三块是训练结束后的 final_demo，用学到的 Q 表把最优路径完整演示出来，方便直观看到最后学到的策略。

我先做的是把 Q 表放进一个智能体类里，也就是 QLearningAgent。这里最核心的是 self.q_table，它是用来保存每个状态下四个动作的分数。动作我用 self.actions 等于 0 1 2 3 来表示，对应环境里 step 里四个方向的移动。我遇到的第一个实际问题是，环境给我的状态是 canvas.coords 的像素坐标，如果直接用像素当状态，会很不稳定，也不方便在表里查找和更新。因为同一个格子应该被当成同一个状态，不然走来走去就像一直在遇到新状态，Q 表就学不起来。为了解决这个，我写了 _state_to_key，把像素坐标换算成格子坐标，再拼成 x_y 这样的字符串当 key。这样 Laura 只要还在同一个格子里，这个 key 就不会变，Q 表更新也能一直落在同一行上，训练才会有积累。配套的我还写了 _ensure_state，用来保证第一次走到某个格子时，Q 表里一定有这一行，不会出现查不到值的问题，而且初始值都设成 0，表示一开始对四个方向都没有偏好。

动作选择这一块我写在 choose_action 里，用的是 epsilon greedy。我的写法是先生成一个 0 到 1 的随机数，然后用 if np.random.rand 大于 self.epsilon 来决定走哪一条分支。这里我把 epsilon 当成探索的概率，所以当随机数小于等于 epsilon 时就探索，直接随机选一个动作。反过来当随机数大于 epsilon 时就利用，也就是从 Q 表这一行里找当前最大的 Q 值对应的动作来走。这样探索的概率就是 epsilon，利用的概率就是 1 减 epsilon。这个设置对我来说比较直观，因为训练刚开始时 Q 表都是 0，如果上来就一直利用，其实也等于在瞎选，而且很可能因为一些偶然的走法就固定住了。保留探索可以让它多试几种走法，然后慢慢把更好的方向区分出来。还有一个细节是并列最大

值的情况，我会把所有最大值动作都找出来，再随机挑一个。因为早期经常四个动作的值都一样，如果每次都选同一个动作，会显得动作选择偏置明显，随机打破平局会更自然一些，也更符合训练阶段到处试的感觉。

Q 值更新我放在 learn 里，这一步我主要注意的是终止状态怎么处理。因为走到宝藏或者踩到陷阱之后回合就结束了，这时就没有下一步可以参考了，所以我的代码里如果 s_ 是 terminal，就直接把目标值设成当前奖励 r。没有结束时就用 r 加上 gamma 乘以下一状态里最大的 Q 值，也就是把下一步可能得到的最好结果考虑进去。更新的时候我用的是 q_predict 加 alpha 乘以 q_target 减 q_predict，这样不会一下子把值跳过去，而是往目标方向挪一点。这里 alpha 控制更新幅度，gamma 控制对后面收益的重视程度。因为迷宫里大多数格子奖励都是 0，只有到宝藏是 1，踩陷阱是负的，所以我希望它别只看眼前一步，而是能把后面能不能走到宝藏也考虑进去，所以 gamma 我设为 0.9。alpha 我设为 0.1，是希望更新不要太猛，训练过程更稳一点，不然可能会出现走法一直抖来抖去的感觉。

训练过程中我还加了 decay_eps，这个是让 epsilon 慢慢变小。我的想法是前期需要探索，不然很可能一直走不到宝藏，或者一直在某些区域绕圈。等试得差不多了，就应该更多利用已经学到的东西，不然每轮都随机走的话，成功率会一直忽高忽低，看起来就像学会了又总是犯错。所以我写成 epsilon 等于 max epsilon_min 和 epsilon 乘 epsilon_decay。我先把 epsilon 设为 0.9，decay_eps 设为 0.995，epsilon_min 设为 0.05。epsilon_decay 让探索概率逐步下降，epsilon_min 给一个下限，避免完全不探索，这样后期还能偶尔试一下别的路，防止过早死在一条不太好的路线里。

把智能体准备好之后，我在 train_q_learning 里把训练循环补齐。每一轮先 reset 拿到初始状态，然后在 while not done 并且步数没有超过 max_steps_per_episode 的循环里重复做三件事。先用 choose_action 选动作，再用 env.step 得到新的状态 s_ 奖励 r 和 done 标志，最后把这一次的经历交给 learn 去更新 Q 表。这样一轮回合结束后，Q 表里就会多一些关于哪些动作更好的信息。为了判断训练是不是在变好，我用 success_hist 记录每一轮有没有走到宝藏，然后用 stable_window 看最近一段回合的平均成功率。我的理解是，单次成功可能是运气，但如果最近一段时间成功率都很高，才比较像是真的学稳定了，所以我用 success_threshold 作为停止条件，这样训练不会只靠跑固定轮数，而是看表现是否已经稳定起来。同时我也会定期在终端打印 episode，epsilon 和 recent_success，这样后面写结果时就能解释为什么前期成功率低，后面会逐渐上升，也能看到 epsilon 下降之后整体会更稳定。

训练结束后我用 final_demo 来演示最终路径，这里我不再用 choose_action 的探索逻辑，而是每一步都用 best_action 选当前状态下 Q 值最大的动作。这样走出来的路线更干净，能直接体现 Q 表最后学到的策略是什么，而不是掺杂训练阶段的随机性。

最后是可视化速度这块，我是一边跑一边才发现需要改的。一开始我直接用原本的渲染节奏去跑训练，发现如果每一步都有比较长的 time.sleep，训练过程会慢得很明显，看起来像一直卡着走。后来我加了 FAST 开关和两套 sleep 参数。训练阶段 FAST 等于 True，就用 RESET_FAST 和 RENDER_FAST 把 sleep 缩短，让训练推进快一些，同时用 render_every 只在部分回合渲染来观察路线变化。训练结束后我再把 FAST 切成 False，用 RESET_SLOW 和 RENDER_SLOW 把 final_demo 放慢，这样最后那条最优路径每一步都更清楚。

3.2 结果的详细分析

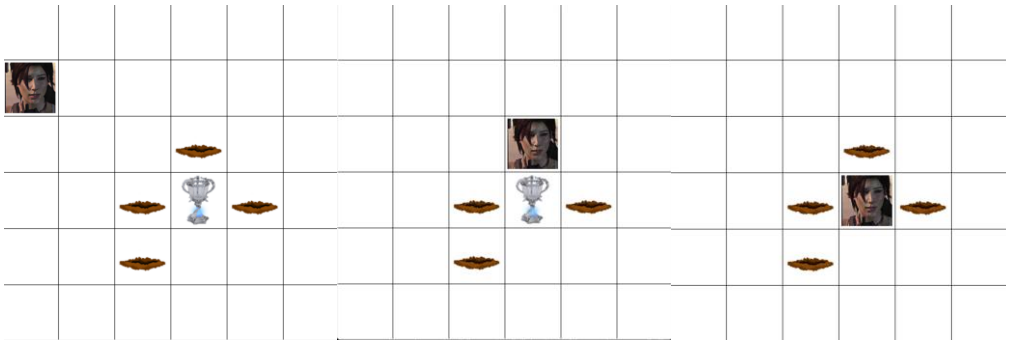

图 1：训练过程中的迷宫界面截图

训练过程中，迷宫窗口里 Laura 的移动本身是可以被观察到的，所以我截取了几张具有代表性的界面截图，当作训练过程会看到什么的例子。图 1 左边那张更像是早期的表现，走法比较散，看起来是在到处试。中间那张是踩到陷阱后这一轮直接结束。右边那张则是后面训练久一点时能走到宝藏的情况。这三张图主要起到一个辅助作用，就是说明训练过程中确实会经历从不稳定到逐渐能成功的各种情况，但真正判断有没有变好，还是得看整体的日志趋势。

```
[Train] episode=   1  epsilon=0.895  steps= 70  recent_success=0.00
[Train] episode=  50  epsilon=0.700  steps=  7  recent_success=0.00
[Train] episode= 100  epsilon=0.545  steps= 25  recent_success=0.13
[Train] episode= 150  epsilon=0.424  steps= 25  recent_success=0.48
[Train] episode= 200  epsilon=0.330  steps= 10  recent_success=0.65
[Train] episode= 250  epsilon=0.257  steps= 10  recent_success=0.70
[Train] episode= 300  epsilon=0.200  steps= 12  recent_success=0.75
[Train] episode= 350  epsilon=0.156  steps= 13  recent_success=0.85
[Train] episode= 400  epsilon=0.121  steps= 10  recent_success=0.87
[Train] episode= 437  epsilon=0.101  steps= 10  recent_success=0.90
[Stop] success rate >= 0.90 over last 60 episodes
[Demo] running greedy policy
[Demo] finished in 10 steps, result=treasure
[Done] close the window to exit
```

图 2：终端训练日志与最终演示结果

训练的整体变化我主要是通过终端日志来看的，结果都能在图 2 里对应到。因为单看某一次成功或失败很容易被随机性影响，所以我更关注几项指标随 episode 的变化趋

势。首先是 epsilon，它会随着 episode 增加逐渐变小，这表示后面随机探索会减少，Laura 会更常按照当前 Q 表里更高的 Q 值去做选择。然后是 recent_success，它是最近一段轮次的平均成功率，比单次结果更能体现“是不是越来越稳定”。从这次输出能看到 recent_success 在前期基本接近 0，到了大概 100 到 200 轮之间开始明显上升，之后继续提高，最后达到 0.90，并触发 Stop success rate >= 0.90 over last 60 episodes，这说明最近 60 轮里到达宝藏的比例已经达到 0.90，所以程序就在这里结束训练。至于 steps，我也会顺带看一下它大概的范围，比如这一轮走得长不长，后期有没有更常出现比较短的路线，但我不会把它当成核心标准，因为 steps 本来就会有波动，尤其在探索还比较多的时候更明显。训练结束后程序进入 Demo，终端打印 finished in 10 steps，result=treasure，可以看到最终演示这一轮里，Laura 用学到的策略从起点走到宝藏，并且用了 10 步完成。

4. 2-step Q-learning 设计与实现

4.1 2-step Q learning 公式推导与符号解释

这里我先完全从题目给的一步更新式出发来推两步更新。一步更新的写法是用当前这一对状态和动作的旧值作为起点，然后往一个目标值靠近一点。把它拆开看会更清楚。先把一步更新里的目标写成

$$G_t^{(1)} = R_{t+1} + \gamma \max_a Q(S_{t+1}, a)。$$

那么更新就可以理解成

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[G_t^{(1)} - Q(S_t, A_t)\right]。$$

也就是旧的 Q不会被直接替换成目标，而是用 α控制每次只往目标方向挪一小段，这样更新会更平滑，不会因为某一次经历就跳得幅度过大。

要改成两步更新，我的想法是继续沿用同样的结构，只是把目标值从看一步变成看两步。一步目标里有一项 $\max_a Q(S_{t+1}, a)$，它代表从下一步状态出发的最好估计。那我现在想把这个估计再往后推一步，也就是把下一步的最好估计写成下一步先拿到的回报，再加上从两步后状态出发的最好估计。用同样的符号表达就是 $max_a Q(S_{t+1}, a)$ 这一项可以用两步的信息近似成

$$R_{t+2} + \gamma \max_a Q(S_{t+2}, a)。$$

把它带回到一步目标 $G_t^{(1)}$里就得到两步目标

$$G_t^{(2)} = R_{t+1} + \gamma[R_{t+2} + \gamma \max_a Q(S_{t+2}, a)]。$$

把括号展开就是

$$G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 \max_a Q(S_{t+2}, a)。$$

最后把目标换成 $G_t^{(2)}$，更新形式仍然保持一致

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma R_{t+2} + \gamma^2 \max_a Q(S_{t+2}, a) - Q(S_t, A_t) \right]。$$

这里每个符号的含义是，S_t 是第 t步所在的状态，A_t 是第 t步选择的动作，R_{t+1} 和 R_{t+2}是接下来两步分别拿到的回报，S_{t+2} 是两步之后到达的状态。参数 α表示每次更新靠近目标的幅度，γ 表示对后续回报的重视程度。两步目标里出现 γ和 γ^2的原因也很直观，就是第 $t+2$步相对更远，所以它在目标里被多乘了一次 γ。

题目里还要求说明什么时候更新 t时刻对应的 $Q(S_t, A_t)$。两步更新的特点是，只有在走完两步以后，R_{t+1}，R_{t+2} 和 S_{t+2}这些信息才完整，所以 t时刻那一步的更新不会立刻发生，而是要等到走到 $t+2$之后再回头更新。还有一个必须考虑的情况是回合可能会提前结束，比如第二步就到宝藏或者踩陷阱，这时就没有后续的 S_{t+2}可以继续估计了，所以两步目标在这种情况下就只保留到两步回报为止，也就是用 $G_t^{(2)} = R_{t+1} + \gamma R_{t+2}$来更新会更合理。

4.2 程序实现与设计思路说明

在把两步更新写进程序时，我最先想清楚的是两件事。第一是两步更新需要等信息凑齐才能做更新，所以训练循环不能再像一步那样走一步就立刻更新这一条经历，而是要能记住上一条经历，然后在下一步到来时把两步拼起来更新前一步。第二是训练里会有很多边界情况，比如刚走一步就结束，或者到最大步数被截断，这些情况下如果只写两步更新，很容易把最后一小段经历漏掉，所以我需要一个补位的更新方式把尾巴收干净。

基于这两个考虑，我在智能体里保留了两种更新。learn_2step 负责做两步目标的更新，它对应的就是 4.1 推出来的 $G_t^{(2)}$。我把输入分成两步的回报 r1 和 r2，再加上两步后的状态 s2，这样目标值就能按同样的结构写出来。这里我单独加了 done2，是因为训练时回合可能在第二步就结束，如果这时还强行去用两步后状态的最大 Q 值，会让目标值的含义变得不清楚，所以我用 done2 把这两种情况分开处理。第二步结束时目标只用 r1 加 gamma 乘 r2，没有结束时目标再加上 gamma 的平方乘两步后状态的最大 Q 值。更新时我仍然用旧值加上 alpha 乘目标减旧值，这个写法我保留下来是因为它和一步更新的思路完全一致，读起来也比较直观，就是每次让当前 Q 值往目标方向靠近一点。

learn_1step 在这里更像是一个收尾工具。两步更新一定要两步齐全才算得出来，但训练里经常会遇到不齐全的情况。比如这一轮刚走一步就结束，那么上一条经历根本凑不

出第二步。又比如达到最大步数时循环退出，最后那一步也可能没等到下一步触发两步更新。如果我不处理这些情况，Q 表就会出现一种不连续的感觉，明明经历过的最后几步没有被写进去，训练的有效信息会被浪费。所以我在这些边界位置用 learn_1step 把剩下的一步补上，而且我让它接收 done，当回合已经结束时就直接用当前回报作为目标，不结束时才去看下一状态的最大 Q 值，这样逻辑上也更顺，不会把不存在的后续硬塞进来。

训练循环这边我用 pending 来解决两步信息的拼接问题。我的思路是每走一步先把这一条经历正常拿到，也就是从当前状态选动作，然后得到下一状态和回报。关键在于更新的顺序，我不急着立刻更新当前这一步，而是先看 pending 里有没有上一条经历。如果 pending 已经存了一条，那说明现在刚好凑齐两步，我就立刻用上一条的状态动作和回报，加上当前这一步的回报和状态，调用一次 learn_2step 更新前一步。更新完之后再把当前这一步存进 pending，让它等待下一步来触发更新。这样从第二步开始基本每走一步都会顺带把前一步更新掉，训练过程也就真的实现了题目要求的在 $t+2$时刻去更新 t时刻的 $Q(S_t, A_t)$。

在终止和截断的处理上我也比较注意不要漏信息。只要当前一步 done1 变成 True，就说明已经到终点或踩陷阱，这时两步链条会被切断，所以我会先把当前这一步用 learn_1step 落一次，让奖励对应到这条经历上，然后结束这一轮并记录是否成功。若是因为走满 max_steps_per_episode 停下，我会在循环结束后检查 pending 是否还留着最后一条没有被两步更新覆盖的经历，如果有就用 learn_1step 再补一次。这样做的目的不是为了让结果看起来更漂亮，而是为了让训练时每一段已经发生的经历尽量都有对应的更新，不然最后几步经常被丢掉会让训练信息不完整。

最后我保留了动作选择的 epsilon greedy 和 epsilon 衰减这一套流程，没有特意去改它。原因是我这次的重点是把更新从一步改成两步，而探索和利用的节奏在我的程序里已经能稳定跑起来了，如果同时大改两块东西，后面出现问题会更难判断到底是哪一块导致的。训练结束后我还是用演示模式去跑学到的策略，主要是为了把最终学到的路线直观地展示出来，也方便我对照训练日志去理解，前期到后期到底发生了哪些变化。

4.3 结果的详细分析

```
[Train] episode=   1  epsilon=0.895  steps= 19  recent_success=0.00
[Train] episode=  50  epsilon=0.700  steps= 60  recent_success=0.02
[Train] episode= 100  epsilon=0.545  steps= 18  recent_success=0.23
[Train] episode= 150  epsilon=0.424  steps= 22  recent_success=0.55
[Train] episode= 200  epsilon=0.330  steps=  9  recent_success=0.73
[Train] episode= 250  epsilon=0.257  steps= 14  recent_success=0.82
[Train] episode= 300  epsilon=0.200  steps= 11  recent_success=0.85
[Train] episode= 312  epsilon=0.188  steps= 16  recent_success=0.90
[Stop] success rate >= 0.90 over last 60 episodes
[Demo] running greedy policy
[Demo] finished in 10 steps, result=treasure
[Done] close the window to exit
```

图 3：2-step Q Learning 终端训练日志与最终演示结果

从这张训练日志截图里可以直观看到几件事。随着 episode 往后走，epsilon 一路从 0.895 慢慢降到 0.188，说明后期随机探索在减少。recent_success 一开始基本接近 0，到了中段开始明显往上走，后面连续几次打印都在提高，最后达到 0.90 并触发停止条件。steps 这一列在不同轮次之间起伏比较大，有的回合走得很久，有的回合步数很少，我更倾向于把它当成训练过程中的正常波动，重点还是看 recent_success 这种更能反映整体趋势的指标。最后 Demo 的输出显示 finished in 10 steps，result=treasure，说明用最终学到的策略可以从起点走到宝藏。

把这份两步更新的日志和一步更新的训练日志放在一起看，我觉得最值得对比的是上升的节奏和稳定的方式。两边都会经历一个从不稳定到逐渐稳定的过程，但在这次两步更新的输出里，recent_success 在中段上升得比较集中，比如从 0.55 到 0.73 再到 0.82 的变化出现在相对连续的区间里，后面也比较顺地到 0.90 并停下。我的理解是，两步更新在一次更新里会把更近的两步结果一起带进去，所以前面那一步更快能得到一个方向感，训练早期就不至于总是用差不多的回报去更新差不多的值，所以趋势看起来更容易拉开。最后两种训练在 Demo 里都能走到宝藏，而且步数都是 10 步，这一点让我更愿意把差别理解成训练过程的收敛速度和稳定性表现不太一样，而不是最终能不能到达宝藏这件事不一样。

5. 总结

这次项目里我先把迷宫宝藏任务整理成一个强化学习问题，把状态，动作和回报这些要素定义清楚，然后在这个框架下实现了一步 Q learning 的训练循环，让 Laura 可以通过一轮一轮的交互去更新 Q 表，并逐渐学到在不同格子里更合适的移动方向。训练过程中我用 epsilon greedy 来平衡探索和利用，并配合 epsilon 的衰减，再结合终端里 recent_success 这类输出去观察训练有没有变得更稳定，最后用演示模式把学到的路线跑一遍，直观看到最终策略的效果。接着我在题目给的一步更新式基础上推导出两步更新的目标形式，并在程序里用 pending 把相邻两步的经历拼起来更新前一步，同时保留一步更新在凑不齐两步信息时做补充更新。整体来看，两种方法最后都能在演示里走到宝藏，而且演示步数一致，但两步更新的训练日志里成功率上升更集中，也更容易更快进入稳定状态。

6. 参考文献

[1]《人工智能原理》课程组. **Lecture 11（课程讲义，PDF）**[Z]. 2025.

[2]《人工智能原理》课程组. **Lecture 12（程讲义，PDF）**[Z]. 2025.

[3]《人工智能原理》课程组. **Lecture 13（程讲义，PDF）**[Z]. 2025.